JavaScript this Keyword

Arrow Functions · Binding · Interview Deep-Dive

1. What is this?

In JavaScript, **this** is a special keyword that refers to the **execution context** — the object that is currently running the code. Its value is **not fixed**; it is determined dynamically at **call-time** (for regular functions) or at **define-time** (for arrow functions).

Context	this value	Example
Global scope (non-strict)	window / global	console.log(this)
Global scope (strict mode)	undefined	'use strict'; console.log(this)
Object method	The object itself	obj.greet()
Arrow function	Lexical (parent scope)	() => this
Constructor (new)	Newly created object	new Person()
Event listener	The DOM element	btn.addEventListener(...)
call / apply / bind	Explicitly passed object	fn.call(obj)

2. Arrow Functions & this

Shorter Syntax

```
// Arrow function syntax
const greet = name => `Hello, ${name}`; // clean & short ■
const add = (a, b) => a + b; // no braces = implicit return
const log = () => console.log('hi'); // no args → empty parens
```

Arrow functions do **NOT** have their own **this**. They inherit it from the **lexical (surrounding) scope** — wherever the arrow was **written**, not where it is called.

```
// Regular vs Arrow — object method
const user = {
name: 'Akshay',
regular: function() { console.log(this.name); }, // 'Akshay' ■
arrow: () => { console.log(this.name); } // undefined ■
};
user.regular(); // 'Akshay' — this = user object
user.arrow(); // undefined — this = global (arrow born outside object)
```

Key Rule: Object {} creates NO new scope. An arrow inside an object literal sees the global/module scope as its parent — NOT the object.

Arrow Functions Are Missing 3 Things

```
// 1. No own `this` → borrows from nearest parent function
// 2. No `arguments` → use ...rest instead
// 3. No `new` → cannot be used as constructor
const fn = (...args) => console.log(args); // ■ rest works
const Foo = () => {};
new Foo(); // TypeError: Foo is not a constructor ■
```

Best Use Cases

Situation	Use Arrow?	Why
Callbacks (setTimeout)	■ YES	Keeps outer this
.map() / .filter()	■ YES	Short, no this needed
React event handlers	■ YES	Preserves component this
Object methods	■ NO	this becomes global
Constructors (new)	■ NO	No prototype, no new
Prototype methods	■ NO	this won't point to instance

---

3. Common this Scenarios — With Output

3a. Regular Function Inside setTimeout — Classic Bug

```
// Problem
const timer = {
count: 0,
start: function() {
setTimeout(function() {
this.count++; // BUG: this = window/undefined
console.log(this.count); // NaN
}, 1000);
}
};
timer.start();
```

```
// Fix with arrow function
const timer = {
count: 0,
start: function() {
setTimeout(() => { // Arrow inherits this from start()
this.count++; // this = timer object
console.log(this.count); // 1
}, 1000);
}
};
timer.start();
```

3b. Class — this in Methods vs Arrow Properties

```
// Class context
class Counter {
constructor() { this.value = 0; }
// Regular method — works fine when called as counter.increment()
increment() { this.value++; console.log(this.value); }
// Arrow property — this is always the instance, even when detached
decrement = () => { this.value--; console.log(this.value); }
}
const c = new Counter();
const inc = c.increment; // detached reference
inc(); // TypeError (strict) or NaN
const dec = c.decrement; // detached reference
dec(); // -1 (arrow remembers instance)
```

3c. React Class Component — Event Handler this

```
// React class component
class Button extends React.Component {
constructor(props) {
super(props);
this.state = { clicked: false };
// Must bind OR use arrow to fix this
this.handleClick = this.handleClick.bind(this); // Option A
}
handleClick() { this.setState({ clicked: true }); } // regular fn
// Option B — class field arrow (no bind needed)
handleClickArrow = () => { this.setState({ clicked: true }); }
render() {
return <button onClick={this.handleClick}>Click</button>;
}
}
```

3d. call / apply / bind — Explicit this

```
// call / apply / bind
function introduce(lang, years) {
console.log(`${this.name} uses ${lang} for ${years} years`);
}
const dev = { name: 'Akshay' };
introduce.call(dev, 'JavaScript', 3); // Akshay uses JavaScript for 3 years
introduce.apply(dev, ['TypeScript', 2]); // Akshay uses TypeScript for 2 years
const boundFn = introduce.bind(dev, 'DAML'); // locks this + first arg
boundFn(5); // Akshay uses DAML for 5 years
```

3e. Chained Array Methods — Arrow Preserves Context

```
// Array methods with this
const team = {
name: 'FullStack',
members: ['Alice', 'Bob', 'Carol'],
greetAll() {
// Arrow in .map() captures 'this' from greetAll()
return this.members.map(m => `${this.name}: ${m}`);
}
};
console.log(team.greetAll());
// ['FullStack: Alice', 'FullStack: Bob', 'FullStack: Carol'] ■
```

3f. Prototype Method — Never Arrow

```
// Prototype methods
function Person(name) { this.name = name; }
// ■ Correct — regular function on prototype
Person.prototype.greet = function() {
return `Hi, I am ${this.name}`;
};
// ■ Wrong — arrow on prototype, this = global
Person.prototype.bye = () => `Bye from ${this.name}`;
const p = new Person('Akshay');
console.log(p.greet()); // 'Hi, I am Akshay' ■
console.log(p.bye()); // 'Bye from undefined' ■
```

---

4. Interview Questions & Answers

Q1: What does `this` refer to in a regular function vs an arrow function?

Regular function: `this` is set at call-time — it depends on HOW the function is called (the object before the dot). Arrow function: `this` is set at define-time — it borrows from the enclosing lexical scope and never changes.

Q2: Why does arrow function `this` become `undefined` inside an object literal?

Object literals {} do not create a new execution scope. When an arrow function is written inside an object, the nearest enclosing scope is either the global scope or the enclosing function. In strict mode (modules, classes) global this is undefined; in non-strict it is window.

Q3: What are the 3 things arrow functions are missing?

1) Own `this` — borrows from lexical scope.

2) `arguments` object — use rest params (...args) instead.

3) `new` / constructor ability — has no prototype, cannot stamp a new object.

Q4: How would you fix a lost `this` in a class event handler?

Three ways: (a) bind in constructor: this.fn = this.fn.bind(this), (b) use a class field arrow: handleClick = () => {...}, (c) use an inline arrow in JSX: onClick={() => this.handleClick()} — though option (b) is preferred in React because it avoids re-creating on every render.

Q5: What is the difference between call, apply, and bind?

All three explicitly set `this`. call(obj, arg1, arg2) invokes immediately with separate args. apply(obj, [arg1, arg2]) invokes immediately with an array. bind(obj, arg1) returns a NEW function with `this` locked — useful for callbacks.

Q6: Can you use an arrow function as a constructor?

No. Arrow functions have no prototype property and no own `this`, so `new ArrowFn()` throws a TypeError: ArrowFn is not a constructor.

Q7: What does `this` equal inside a `setTimeout` callback?

With a regular function callback, `this` loses its context and becomes window (non-strict) or undefined (strict). Fix: use an arrow function so it inherits `this` from the outer scope, or store const self = this before the setTimeout.

Q8: Predict the output — tricky question

const obj = { x: 10, fn: function() { const inner = () => this.x; return inner(); } };

console.log(obj.fn()); // 10 ■ — inner arrow captures this from fn(), which is obj

---

5. Quick Cheat Sheet

```
// REGULAR FUNCTION → this = who called it
obj.method() // this = obj
fn() // this = window or undefined (strict)
new Fn() // this = new blank object
fn.call(obj) // this = obj (explicit)
// ARROW FUNCTION → this = where it was written
const arrow = () => {} // this = outer scope, locked forever
// .call/.apply/.bind DO NOTHING to arrow this
// GOTCHAS
const { method } = obj; method(); // loses this! ■
setTimeout(obj.method, 0); // loses this! ■
setTimeout(() => obj.method(), 0); // keeps this! ■
```

Golden Rule: If you need `this` from the parent scope — use ARROW. If you need `this` to be the caller — use REGULAR function.